

Project : Swasthy Queue

1. Introduction

1.1 Purpose

To provide a digital transparency layer over the physical hospital queue system, allowing patients to monitor their wait time remotely and hospital staff to manage patient flow efficiently.

1.2 Scope

The system involves a patient-facing mobile-responsive web app, a doctor/staff dashboard for calling the "Next" patient, and an admin panel for department management.

2. Overall Description

2.1 Product Perspective

SwasthyaQueue is a Real-Time SaaS platform. It utilizes WebSockets (Socket.io) for instant updates, ensuring that when a doctor clicks "Next," every patient's phone screen updates immediately.

2.2 User Classes

- Patients: Can join a queue via QR code, view live token numbers, and receive notifications.
- Hospital Staff/Reception: Register walk-in patients and assign them to specific doctors.
- Doctors: View their daily list and trigger "Next Patient" or "On Break" status.
- Hospital Admin: Analyze peak hours and department-wise average wait times.

3. System Features (Functional Requirements)

3.1 Digital Token Generation

- Description: Patients can generate a digital token by scanning a QR code placed outside the OPD room or via the hospital portal.
- Logic: The system generates a unique Token_ID based on the department and timestamp.

3.2 Live Queue Tracker (The Core Feature)

- Description: A real-time dashboard showing:
 - Current Token being served.
 - User's own Token number.

- Estimated Wait Time (EWT): Calculated as (Remaining Patients) x (Avg. Checkup Time).

3.3 Doctor's "Patient-Flow" Console

- Description: A simplified interface for doctors.
- Features:
 - "Call Next" button.
 - "No Show" button (moves patient to the end or cancels).
 - "Emergency Override" to jump the queue.

3.4 Data Visualization & Analytics

- Description: Admin dashboard showing wait-time trends.
- Visuals: Bar charts showing "Busiest Hours of the Day" and "Average Consultation Time per Doctor."

4. Database

5. Non-Functional Requirements

- Real-time Synchronization: Token updates must reflect on the patient's UI in less than 500ms.
- Scalability: Must handle 1,000+ concurrent WebSocket connections per hospital.
- Accessibility: Mobile-first design; must work on low-end smartphones and slow 4G/3G networks.

6. Advanced Features

1. SMS/WhatsApp Alerts: Notifies the patient when there are only 3 people ahead of them.
2. Voice Announcement Integration: A "Call" button that triggers a browser-based Text-to-Speech announcement in the waiting hall.
3. Predictive Analytics: Uses historical data to tell a patient, "Monday mornings are 40% busier; try visiting on Tuesday."

7. Why Recruiters Will Like It

- Real-Time Tech: Proves you can work with WebSockets/Socket.io, which is a high-level skill.
- Algorithm Logic: Calculating the "Estimated Wait Time" involves dynamic variables and logic.
- Social Impact: It solves a visible, daily problem in India—overcrowded hospital waiting rooms.

